

TOAE209/TOAH209 – Design and Analysis of Algorithms

Week 15: Theoretical Questions

Foreign Trade University

Instructions

Part A contains 18 questions requiring written explanations or short proofs. Part B is a 10-question quiz (true/false and multiple choice). Attempt every question on your own before consulting Part C (Solutions) at the end of this document.

Part A: Theory Questions

- A1.** Define a *randomized algorithm*. Explain the difference between a **Las Vegas** algorithm and a **Monte Carlo** algorithm in terms of what is guaranteed (correctness) and what is random (running time). Give one example of each from this week.
- A2.** State *linearity of expectation* precisely. Why is it so useful that it holds **even when the random variables are not independent**? Illustrate with the phrase “decompose into indicators, then sum.”
- A3.** Define an *indicator random variable* $\mathbf{1}_A$ for an event A . Prove that $\mathbb{E}[\mathbf{1}_A] = \Pr[A]$, and explain why this identity is the bridge that lets us compute probabilities by computing expectations (and vice versa).
- A4.** State the *union bound* and prove it for two events from the inclusion-exclusion identity $\Pr[A \cup B] = \Pr[A] + \Pr[B] - \Pr[A \cap B]$. Why does the bound require no independence assumption?
- A5.** In the contention-resolution model, n identical processes each attempt to access a shared resource with probability p per round; an attempt succeeds only if exactly one process attempts that round. Derive the probability that a fixed process P_i succeeds in a given round, and show it is maximized at $p = 1/n$.
- A6.** Continuing Q5 with $p = 1/n$: show that P_i succeeds in a given round with probability at least $\frac{1}{en}$, using $(1 - 1/n)^{n-1} \geq 1/e$. Then bound the probability that P_i has *not* succeeded after t rounds.
- A7.** Using the bound from Q6 and the union bound, prove that after $t = \lceil 2en \ln n \rceil$ rounds, all n processes have succeeded with probability at least $1 - 1/n$. Identify exactly where the union bound is applied.
- A8.** State randomized QuickSort. Explain why the analysis counts *comparisons*, and state the key structural fact that any two elements are compared *at most once* over the whole execution. Why is that fact true?

- A9.** Let $z_1 < z_2 < \dots < z_n$ be the sorted order. Prove that in randomized QuickSort, $\Pr[z_i \text{ and } z_j \text{ are compared}] = \frac{2}{j-i+1}$ for $i < j$. (Hint: consider the first element of $\{z_i, \dots, z_j\}$ chosen as a pivot.)
- A10.** Using Q9 and linearity of expectation, prove that the expected number of comparisons in randomized QuickSort is at most $2n \ln n = O(n \log n)$. Where does the harmonic-series bound enter?
- A11.** State randomized Quickselect for finding the k -th smallest element. Sketch the argument that its expected running time is $O(n)$ (define a “good” pivot, bound the expected number of pivots per phase, and sum the geometric series of shrinking subarray sizes).
- A12.** Describe Karger’s contraction algorithm for global min-cut: what a *contraction* does to an edge, why *parallel edges are kept but self-loops deleted*, and when the algorithm stops.
- A13.** In Karger’s algorithm, fix a particular min-cut C^* of size k . Prove that every vertex has degree $\geq k$, and conclude that at any stage with m super-vertices there are at least $mk/2$ edges. Why does this matter for bounding the chance of contracting a C^* -edge?
- A14.** Prove that a single run of Karger’s algorithm returns a fixed min-cut C^* with probability at least $1/\binom{n}{2}$. Show the telescoping product explicitly.
- A15.** Karger’s single-run success probability is only $\approx 2/n^2$. Explain *amplification*: how many independent repetitions T suffice to find the min-cut with probability $\geq 1 - 1/n$, and prove the bound using $1 - x \leq e^{-x}$.
- A16.** State the reservoir-sampling algorithm for selecting k items from a stream of unknown length. Prove that every item ends up in the reservoir with probability exactly k/n after n items are seen. Point out where the product telescopes.
- A17.** Define a *universal* family of hash functions. For the family $h_{a,b}(x) = ((ax + b) \bmod p) \bmod m$, use indicator variables and linearity of expectation to show that the expected number of colliding pairs when hashing n keys into m buckets is at most $\binom{n}{2}/m \approx n^2/(2m)$.
- A18.** *Discussion (open-ended).* The Miller–Rabin primality test is Monte Carlo: it never errs on a prime, but a composite may “fool” a random witness with probability $\leq 1/4$. Explain how to (a) drive the error below any target δ by repetition, and (b) why a Monte Carlo algorithm with a *checkable* answer can be converted into a Las Vegas algorithm. Could primality testing be made Las Vegas this way? Discuss.

Part B: Quiz

- B1. (True/False)** A Las Vegas algorithm may return an incorrect answer but always runs within a fixed time bound.
- B2. (Multiple choice)** Linearity of expectation, $\mathbb{E}[\sum_i X_i] = \sum_i \mathbb{E}[X_i]$, holds:
- (a) Only if the X_i are independent
 - (b) Only if the X_i are identically distributed
 - (c) For any random variables, independent or not
 - (d) Only for indicator random variables
- B3. (Short answer)** In randomized QuickSort on n distinct elements, what is the probability that the i -th and j -th smallest elements ($i < j$) are ever compared?
- B4. (True/False)** The expected number of comparisons made by randomized QuickSort is $O(n \log n)$ on *every* input, with no worst-case bad inputs.
- B5. (Multiple choice)** Karger's contraction algorithm, on a graph with n vertices, returns a fixed global min-cut in a single run with probability at least:
- (a) $1/n$
 - (b) $1/\binom{n}{2}$
 - (c) $1/2$
 - (d) $1/n!$
- B6. (True/False)** Running Karger's algorithm $\Theta(n^2 \log n)$ times and keeping the best cut finds the global min-cut with high probability.
- B7. (Short answer)** In reservoir sampling of k items from a stream, with what probability is the t -th arriving item ($t > k$) accepted into the reservoir at the moment it arrives?
- B8. (Multiple choice)** The union bound states that $\Pr[\bigcup_i A_i]$ is:
- (a) Exactly $\sum_i \Pr[A_i]$
 - (b) At most $\sum_i \Pr[A_i]$
 - (c) At least $\sum_i \Pr[A_i]$
 - (d) Equal to $\prod_i \Pr[A_i]$
- B9. (Short answer)** For a universal hash family hashing n keys into m buckets, approximately how many colliding pairs are expected?
- B10. (True/False)** The Miller–Rabin primality test may report a prime as “composite.”

Part C: Solutions

Part A

- A1.** A **randomized algorithm** may consult a source of random bits (a random number generator) during execution, so its behaviour on a fixed input can vary from run to run. A **Las Vegas** algorithm is *always correct*; only its running time is random (we analyze *expected* time) – e.g. randomized QuickSort and Quickselect. A **Monte Carlo** algorithm runs in bounded time but may be *wrong with bounded probability* (we analyze success probability, then amplify) – e.g. Karger’s min-cut and Miller–Rabin. Slogan: Las Vegas gambles with time, Monte Carlo gambles with correctness.
- A2. Linearity of expectation:** for any random variables $X_1, \dots, X_n$ and constants a_i , $\mathbb{E}[\sum_i a_i X_i] = \sum_i a_i \mathbb{E}[X_i]$. It holds *without any independence assumption* because expectation is just a weighted sum over outcomes, and summation can be reordered regardless of how the variables correlate. This is exactly what makes it powerful: to compute the expectation of a complicated quantity, we *decompose it into a sum of indicator variables* (one per “elementary event”), compute each easy probability $\mathbb{E}[\mathbf{1}_A] = \Pr[A]$, and sum – never needing to understand how the events interact.
- A3.** $\mathbf{1}_A$ equals 1 when A occurs and 0 otherwise. Then $\mathbb{E}[\mathbf{1}_A] = 1 \cdot \Pr[A] + 0 \cdot \Pr[\bar{A}] = \Pr[A]$. This identity is the bridge between probability and expectation: it lets us compute a probability by setting up an expectation, and (via linearity) compute the expectation of a sum of events as a sum of their probabilities – the engine behind the QuickSort, reservoir-sampling, and hashing analyses.
- A4. Union bound:** $\Pr[\bigcup_i A_i] \leq \sum_i \Pr[A_i]$. For two events, from inclusion-exclusion $\Pr[A \cup B] = \Pr[A] + \Pr[B] - \Pr[A \cap B] \leq \Pr[A] + \Pr[B]$, since $\Pr[A \cap B] \geq 0$. The general case follows by induction. No independence is needed because we only ever *drop* the (non-negative) intersection terms, which is valid for any events.
- A5.** Process P_i succeeds in a round iff it attempts (probability p) and *no other* process attempts (each of the other $n - 1$ stays silent, probability $(1 - p)^{n-1}$), so $\Pr[\text{success}] = p(1 - p)^{n-1}$. Differentiating $f(p) = p(1 - p)^{n-1}$ and setting $f'(p) = (1 - p)^{n-1} - p(n - 1)(1 - p)^{n-2} = 0$ gives $(1 - p) = p(n - 1)$, i.e. $1 = pn$, so $p = 1/n$ maximizes the success probability.
- A6.** With $p = 1/n$, $\Pr[\text{success in a round}] = \frac{1}{n}(1 - \frac{1}{n})^{n-1} \geq \frac{1}{n} \cdot \frac{1}{e} = \frac{1}{en}$, using $(1 - 1/n)^{n-1} \geq 1/e$. Since rounds are independent, the probability P_i fails every one of the first t rounds is at most $(1 - \frac{1}{en})^t \leq e^{-t/(en)}$ (using $1 - x \leq e^{-x}$).
- A7.** Set $t = \lceil 2en \ln n \rceil$. Then $\Pr[P_i \text{ not yet succeeded}] \leq e^{-t/(en)} \leq e^{-2 \ln n} = 1/n^2$. The bad event “some process has not succeeded” is the union of the n events “ P_i not yet succeeded.” By the **union bound**, its probability is at most $n \cdot 1/n^2 = 1/n$. Hence with probability $\geq 1 - 1/n$ all processes have succeeded – the union bound is applied exactly at the step combining the n per-process failure probabilities.
- A8. Randomized QuickSort:** pick a pivot uniformly at random, partition into elements smaller and larger, recurse on each side, concatenate. We count *comparisons* because they dominate the running time (partitioning is comparison-driven). Key fact: two elements are compared *at most once*, because a comparison occurs only between a pivot and a non-pivot, and once

an element is chosen as a pivot it is removed from all recursive subproblems, so it never gets compared to anyone again.

A9. Consider $Z_{ij} = \{z_i, z_{i+1}, \dots, z_j\}$, the $j - i + 1$ elements between z_i and z_j inclusive. Track the *first* element of Z_{ij} chosen as a pivot. If it is z_i or z_j , then at that moment z_i and z_j are still in the same subarray, so the pivot is compared to the other – they *are* compared. If it is any interior z_k ($i < k < j$), then z_i and z_j are split into different subarrays and never compared. Because pivots are uniform, the first pivot from Z_{ij} is equally likely to be any of the $j - i + 1$ elements; exactly 2 of them (z_i, z_j) cause a comparison. Hence the probability is $\frac{2}{j-i+1}$.

A10. Let $X = \sum_{i < j} X_{ij}$ be the total number of comparisons, X_{ij} the indicator that z_i, z_j are compared. By linearity, $\mathbb{E}[X] = \sum_{i < j} \frac{2}{j-i+1}$. Substituting $k = j - i + 1 \in \{2, \dots, n\}$ and over-counting,

$$\mathbb{E}[X] \leq \sum_{i=1}^n \sum_{k=2}^n \frac{2}{k} = 2n \sum_{k=2}^n \frac{1}{k} \leq 2n \ln n,$$

where the final inequality is the **harmonic-series bound** $\sum_{k=1}^n 1/k \leq 1 + \ln n$. So $\mathbb{E}[X] = O(n \log n)$.

A11. Quickselect: pick a random pivot, partition, and recurse only into the *one* side containing the k -th element (or return the pivot if it is the k -th). *Expected $O(n)$:* call a pivot *good* if it falls between the 25th and 75th percentiles – probability $\geq 1/2$. A good pivot shrinks the surviving subarray to $\leq 3/4$ of its size. Group steps into phases ending at each good pivot; the expected number of pivots per phase is ≤ 2 (geometric, success prob. $\geq 1/2$), and sizes shrink by a factor $\geq 3/4$ each phase. Total expected work $\leq \sum_{j \geq 0} 2c(3/4)^j n = 8cn = O(n)$, the geometric series converging.

A12. A **contraction** of edge (u, v) merges u and v into a single super-vertex; all other edges incident to u or v now attach to the merged vertex. **Parallel edges are kept** because the multiplicity of edges between super-vertices is exactly what records how many original edges cross a potential cut – discarding them would corrupt the cut count. **Self-loops are deleted** because an edge whose both endpoints have been merged into the same super-vertex can never cross any cut. The algorithm **stops when only two super-vertices remain**; the edges between them form the returned cut.

A13. If some vertex v had degree $< k$, then the cut separating $\{v\}$ from the rest would have size $< k$, contradicting that the min-cut is k ; so every vertex has degree $\geq k$. At a stage with m super-vertices, the sum of degrees is $\geq mk$, and since each edge contributes to two degrees, the number of edges is $\geq mk/2$. This matters because it *lower-bounds the total number of edges*, which *upper-bounds* the chance that a uniformly random edge is one of the k cut edges: $k/(mk/2) = 2/m$.

A14. Fix a min-cut C^* of size k . The algorithm returns C^* iff it never contracts a C^* -edge. At step i there are $n - i + 1$ super-vertices, hence $\geq (n - i + 1)k/2$ edges, of which k are in C^* ; so the chance of contracting a C^* -edge is $\leq \frac{2}{n-i+1}$. Thus

$$\Pr[C^* \text{ survives}] \geq \prod_{i=1}^{n-2} \left(1 - \frac{2}{n-i+1}\right) = \prod_{i=1}^{n-2} \frac{n-i-1}{n-i+1} = \frac{n-2}{n} \cdot \frac{n-3}{n-1} \cdots \frac{2}{4} \cdot \frac{1}{3} = \frac{2}{n(n-1)} = \frac{1}{\binom{n}{2}},$$

the product telescoping (each numerator cancels a denominator two steps later).

A15. Amplification: run the algorithm T times independently and keep the smallest cut found. Each run misses C^* with probability $\leq 1 - 1/\binom{n}{2}$, so all T miss with probability $\leq (1 - 1/\binom{n}{2})^T \leq e^{-T/\binom{n}{2}}$ (using $1 - x \leq e^{-x}$). Choosing $T = \binom{n}{2} \ln n$ makes this $\leq e^{-\ln n} = 1/n$. Hence with probability $\geq 1 - 1/n$ some run finds C^* and the best-of- T answer is the min-cut. This is $\Theta(n^2 \log n)$ runs.

A16. Reservoir sampling: keep the first k items; for each later item x_t ($t > k$), with probability k/t replace a uniformly random reservoir element with x_t . *Claim:* item x_t ends in R with probability k/n . It is in R right after step t with probability k/t (for $t \leq k$ this is 1). For each later step $s > t$, x_t is evicted only if step s replaces (prob. k/s) and picks x_t 's slot (prob. $1/k$), i.e. with probability $1/s$; it survives with probability $(s-1)/s$. Therefore

$$\Pr[x_t \in R] = \frac{k}{t} \prod_{s=t+1}^n \frac{s-1}{s} = \frac{k}{t} \cdot \frac{t}{n} = \frac{k}{n},$$

the product telescoping. Since every item is equally likely (k/n), R is a uniform random k -subset.

A17. A family $\mathcal{H}$ is **universal** if for all distinct $x \neq y$, $\Pr_{h \in \mathcal{H}}[h(x) = h(y)] \leq 1/m$. For the n keys, let X_{xy} indicate that the pair (x, y) collides. By linearity, the expected number of colliding pairs is

$$\mathbb{E}\left[\sum_{x < y} X_{xy}\right] = \sum_{x < y} \Pr[h(x) = h(y)] \leq \binom{n}{2} \cdot \frac{1}{m} = \frac{n(n-1)}{2m} \approx \frac{n^2}{2m}.$$

The $(ax+b \bmod p) \bmod m$ family with random $a \in \{1, \dots, p-1\}, b \in \{0, \dots, p-1\}$ is universal, so this bound applies.

A18. (a) Each independent Miller–Rabin witness exposes a composite with probability $\geq 3/4$ (fails to expose with prob. $\leq 1/4$); after r independent witnesses the probability a composite slips through is $\leq (1/4)^r$, which is below any target δ for $r \geq \log_4(1/\delta)$. (b) A Monte Carlo algorithm whose answer can be *efficiently verified* can be made Las Vegas: repeat until the verifier accepts – the output is then always correct, and the (now random) running time is the expected number of trials times the per-trial cost. For *primality*, the situation is subtle: a single “composite” verdict is certain (a witness is a certificate of compositeness), but a “prime” verdict from Miller–Rabin is not directly checkable in the same cheap way. Historically, deterministic polynomial-time primality (AKS, 2002) settled the decision problem outright; randomized Las Vegas *primality proving* (e.g. ECPP) also exists but is more involved. So yes, primality can be decided/proved without one-sided error, just not by trivially wrapping Miller–Rabin in a cheap verifier.

Part B

B1. False. That describes a *Monte Carlo* algorithm. A Las Vegas algorithm is always correct; its *running time* is the random quantity.

B2. (c) For any random variables, independent or not. This is precisely what makes linearity of expectation so useful.

B3. $\frac{2}{j-i+1}$ (probability that the first pivot drawn from $\{z_i, \dots, z_j\}$ is one of the two endpoints).

- B4. True.** Random pivoting removes any worst-case input: the $O(n \log n)$ bound is on the *expected* comparisons for every fixed input.
- B5. (b)** $1/\binom{n}{2}$, equivalently $\frac{2}{n(n-1)}$.
- B6. True.** $T = \binom{n}{2} \ln n = \Theta(n^2 \log n)$ repetitions drive the failure probability to $\leq 1/n$.
- B7.** k/t – the t -th item ($t > k$) is accepted into the reservoir with probability k/t when it arrives.
- B8. (b) At most $\sum_i \Pr[A_i]$.** The union bound is an upper bound; equality holds only when the events are disjoint.
- B9.** About $\frac{n^2}{2m}$ (precisely $\binom{n}{2}/m$) colliding pairs in expectation.
- B10. False.** Miller–Rabin has *one-sided* error: it never misclassifies a prime (it always reports a prime as “probably prime”); only a composite can be reported as “probably prime.”